

OILWISE

Current Product & Technical Architecture Overview

1. Current Product Direction

OILWISE is currently a developing vehicle-focused product architecture rather than a fully defined product specification. The direction so far is an application where authenticated users own and manage private vehicle-related data, preferences, and eventually records associated with their vehicles.

Current positioning: Your vehicle data, all in one place.

The technical foundation is considerably more defined than the final business feature set. The current work is establishing authentication, authorization, user ownership, data relationships, routing, and a scalable monorepo structure before the full vehicle domain is modeled.

2. Architecture

OILWISE is being built as a pnpm/Turborepo monorepo with a Next.js frontend and Convex backend.

```
OILWISE apps/ web/ Next.js 16 frontend packages/ backend/ Convex backend
```

The primary responsibility split is:

- **Next.js** - frontend, UI, pages, and server-side route protection.
- **Clerk** - authentication, identity, sessions, and authentication-related account data.
- **Convex** - application backend, database, realtime data, business logic, and data authorization.

3. User Identity Model

Clerk is the authentication identity source. Convex maintains its own application-level user document and links it to Clerk through the Clerk user ID.

```
Clerk user user_xxxxx | v users.clerkUserId | v Convex users._id | v OILWISE application data
```

This keeps external authentication identity separate from OILWISE's internal database identity.

Relationships between OILWISE tables should generally use the Convex user document ID rather than copying Clerk IDs throughout the database.

4. Current User Model

The Convex user model has evolved toward fields such as:

```
users = { clerkUserId, username, email, name?, has_img?, img_url?, preferred_distance_unit?, phone_number? }
```

Clerk should own authentication-related identity data such as passwords, sessions, authentication email/phone methods, and account security. Convex should own OILWISE-specific preferences, relationships, vehicles, records, and other product data. Some Clerk profile fields may be mirrored into Convex when useful.

5. Clerk-to-Convex Synchronization

The current application has used an `ensureUserProfile`-style mechanism to create or refresh the corresponding Convex user after Clerk authentication. This works as a fallback but is not automatic synchronization.

For a stronger production architecture, Clerk webhooks are the intended direction for changes such as user creation, profile updates, image changes, and deletion. That allows Clerk-managed UI changes to propagate to Convex without requiring a page refresh or a frontend synchronization component.

6. Authentication & Authorization

The application is moving beyond simple signed-in/signed-out checks and toward role-based authorization. Current concepts include anonymous users, authenticated users, and administrators.

The intended security split is important: Next.js protects access to pages and routes, while Convex independently protects the underlying data and mutations. Hiding a page or button is not considered sufficient authorization.

```
Example server-side UI pattern:
```

7. User-Owned Data

The current numbers table is a prototype for establishing the ownership model that future OILWISE entities can follow.

```
numbers = { value, userId: Id<"users"> } .index("by_user_id", ["userId"])
```

The frontend does not decide which user owns a record. Convex derives the authenticated Clerk identity, resolves the corresponding Convex user through `getAuthenticatedUser(ctx)`, and uses that user's internal `_id` for ownership.

```
Clerk session -> getAuthenticatedUser(ctx) -> Convex user -> users._id -> owned application records
```

8. Data Modeling Philosophy

Small values that are naturally part of a user profile can remain embedded directly on the user document, such as language, distance-unit preference, or compact settings.

Entities that can grow, be independently queried, paginated, sorted, referenced, or modified should generally have their own tables and reference the owning user through `userId`. Examples include vehicles, posts, messages, orders, records, maintenance history, and similar resources.

```
Rule of thumb: One small thing / preference -> nested field or object  
Many things with their own lifecycle -> separate table + userId
```

9. Indexing Strategy

Indexes are being designed around application read patterns. For example, numbers are indexed by `userId` because a common query is "give me the records belonging to the authenticated user." Indexes describe how data should be found efficiently; they are not a security mechanism. Authorization remains enforced in Convex queries and mutations.

10. Current UI

The UI is intentionally basic while the architecture is being established. Current or experimental areas include:

- Header and footer navigation
- Home page with authentication-aware calls to action
- Clerk sign-in/sign-up and UserButton integration
- OILWISE profile area and Clerk UserProfile account-management routes
- Internal/admin dashboard experiments
- Numbers form and user-scoped numbers list

11. Current Clerk Trade-offs

Clerk provides authentication, session management, account UI, profile editing, and identity integration with Convex. During development, several trade-offs have become clear: Clerk profile changes do not automatically synchronize into Convex without an explicit mechanism such as webhooks; prebuilt Clerk UI carries Clerk branding on the free production tier; and some production authentication features such as SMS may introduce additional pricing considerations.

12. What Is Still Undefined

The final OILWISE product model is still being defined. The architecture supports user-owned vehicle data, but the exact V1 domain has not yet been finalized. A likely future model may involve a user owning one or more vehicles with related mileage, fuel, maintenance, expenses, documents, reminders, or similar records, but these should not yet be treated as finalized requirements.

Current status: OILWISE has a strong emerging technical foundation for authentication, ownership, authorization, realtime data, and scalable data modeling. The next major step is defining the concrete OILWISE V1 product domain and the vehicle-related entities that will replace or build upon the current prototype data.